

Optimizely CMS 12 (PaaS) – Developer Training Test

Theme: "Opti Blog" - build a working blog application

1. Instructions

1. Start from a clean **Optimizely CMS 12** ASP.NET Core 6/8 project. The Alloy template is allowed, but all blog functionality must be your own code - not renamed Alloy artefacts.
2. Use the database and **Search & Navigation** developer index provided by your trainer.
3. Commit to Git with meaningful, incremental commits.
4. You may use the official Optimizely documentation and your own notes. You may not copy an existing blog accelerator or add-on wholesale.
5. **Must** use static assets including css, script provided by Examiner:
 - Static assets will be copied to wwwroot folder
 - Follow current demo html structure to integrate with FE correctly:

Include this ones to refer js, css file

```
<script type="module" crossorigin src="/blog-landing-page.js"></script>
<link rel="stylesheet" crossorigin href="/blog-landing-page.css">
```

Based on this generated html, integrate with your BE

```
<header class="site-header">
  <div class="container site-header__inner">
    <a class="site-header__logo" href="/index.html">Opti Blog</a>
    <nav class="site-header__nav" aria-label="Primary">
      <ul role="list">
        <li><a href="/index.html">Home</a></li>
        <li><a href="/blog-landing-page.html">Blog</a></li>
        <li><a href="/blog/author/jane-doe">Authors</a></li>
        <li><a href="/site-search.html">Search</a></li>
      </ul>
    </nav>
  </div>
</header>
```

For example:

```
<header class="site-header">
  <div class="container site-header__inner">
```

```

<a class="site-header__logo" href="@Model.HomePageUrl">Opti Blog</a>
<nav class="site-header__nav" aria-label="Primary">
  <ul role="list">
    @foreach(var menuItem in Model.Menu)
    {
      <li><a href="@menuItem.Link">@menuItem.Text</a></li>
    }
  </ul>
</nav>
</div>
</header>

```

- To run mock(sample pages)
 - Install/Use any tools can run static: VSCode, serve, nginx, to serve this assets folder. If using serve: `serve wwwroot`, it will run port 3000
 - Open <http://localhost:3000>, it contains pages that are mentioned in the exercise

6. Submit:

- Git repository (or zip including `.git`)
- `README.md` - how to run it, what is complete, what is not, known issues, and the written justifications requested below
- A content export, **or** a working content-seeding job (Task 5)

Scope note: this test targets **Search & Navigation** on CMS 12. Solutions built against Optimizely Graph will not be accepted for the indexing and search tasks.

2. Domain model

```

Start
├─ Site settings      - single instance per site (Task 4)
├─ Blog              - landing, search and filtering
│   └─ Blog posts
│   └─ Authors        - NOT pages (Task 8)

```

Type	Kind	Properties
SiteSettings	Only Single instance per site	SiteName, DefaultHeader, DefaultFooter, RootCatalog
HeaderBlock / FooterBlock	Block	Logo, NavigationLinks / Columns, CopyrightText
BlogListPage	Page	Heading, Intro (XhtmlString), PageSize (int), MainContentArea

Type	Kind	Properties
BlogPostPage	Page	Title, Summary (string, max 300), MainBody (XhtmlString), PublishDate, Author (string), Tags (IList<string>), HeroImage, RelatedPosts (ContentArea), InternalNotes (string, editorial use only)
BlogTeaserBlock	Block	Post, Kicker
AuthorProfileBlock	Block	AuthorProfileBlock has Slug, Name, Bio, PostCount (Auto updated after a blog is published)

All page types must inherit from a common base page type.

TASK 1 - Content types and templates

Implement **BlogListPage**, **BlogPostPage** and **BlogTeaserBlock** with working templates.

1. Each content type must be fully and correctly described to the CMS, and must remain stable across a database rebuild.
2. **BlogPostPage** must only be creatable beneath **BlogListPage**.
3. **Summary** must be limited to 300 characters, enforced in the editor, and render as plain text.
4. **BlogListPage** must list its blog posts newest first, paged according to **PageSize**.
5. Properties must be organised so the editing experience is logical rather than one flat list.

TASK 2 - TinyMCE custom styles

1. On **BlogPostPage.MainBody**, editors must be able to apply the following styles from the editor's styles dropdown:

Style shown to the editor	Applies to	Resulting class
Primary Button	a link	btn btn-primary
Secondary Button	a link	btn btn-secondary
Lead Paragraph	a paragraph	lead
Pull Quote	a block quote	pull-quote

2. **This must be achieved without writing any JavaScript.** No custom editor plug-in, no script file, no client-side configuration.
3. The style names must appear to editors as the readable labels above - not as internal identifiers.
4. The four styles must render with their correct visual appearance **inside the editor**, matching how they appear on the published page.
5. **BlogListPage.Intro** must use a deliberately reduced editor: basic emphasis, linking and undo/redo only, a shorter fixed height. Configuring this must not alter the editor used by any other property.

TASK 3 - Display options

1. Provide editors with three layout choices for items placed in a content area: full width, half width, and card.
 2. Each choice must show a readable name in the editor interface.
 3. The chosen layout must affect the rendered markup so the item is visibly laid out at that width.
 4. **BlogTeaserBlock** must render using a **different view** when placed with the card layout.
-

TASK 4 - Site settings, header and footer

4a. Site settings

Create a settings (page is under Start page) container holding **SiteName**, **DefaultHeader**, **DefaultFooter** and **AuthorProfileFolder** (the Asset Folder containing all AuthorProfileBlocks).

1. Exactly **one** instance may exist per site. Creating a second must be impossible through the editor interface.
2. The rest of the application must read settings through a single abstraction. Views and controllers must not load the settings content directly.
3. Resolution must work correctly if a second site is added to the same installation.
4. Settings must be cached, and the cache must be invalidated when the settings are republished.

4b. Header and footer behavior

Every page must support three behaviours, **configured independently for the header and for the footer** - a page may override its header while still inheriting its footer:

Behavior	Result
Inherit	Use the site default. This is the behavior when an editor has configured nothing.
Override	Use a header/footer chosen on that page instead of the site default.
Hidden	Render no header/footer at all, and no leftover empty markup.

TASK 5 - Creating and updating content in code

1. Write a scheduled job that creates the blog structure (and the site settings, if absent) plus **25** blog posts with varied publish dates, authors and tags. Running it a second time must not create duplicates.
 2. Write a second scheduled job that finds every blog post, normalises all tags to lower case with surrounding whitespace removed, and republishes **only** those posts that actually changed.
 3. Both jobs must run correctly without an interactive user.
 4. Both jobs must report progress while running, be stoppable, and return a summary that tells an administrator what happened.
 5. Both jobs must appear in the CMS administration interface.
-

TASK 6 - Search indexing conventions

6a. Computed values

Index the following values against each blog post. **None of them may be added as properties on the content model** - the content type must remain unchanged by this task.

Value	Description
Reading time	Estimated minutes to read the body, based on word count, minimum 1
Plain text body	The body content with all HTML markup removed
Age in days	How many days ago the post was published

6b. Controlling what is indexed

1. Images and video must **not** be indexed.
2. Uploaded PDF and Word documents **must** be indexed and remain findable - see Task 7b.
3. Create one utility page type (for example a login or search page) and prevent it from being indexed at all.
4. A blog post with a publish date in the future must not be indexed, even when published.

TASK 7 - Search and boosting

7a. Blog search

On **BlogListPage**, build a search over blog posts only:

1. Free-text search across the title, summary and body text.
2. Results must respect publication state, deleted content and the visitor's read access.
3. Paged results, ordered by relevance, with a toggle to order by newest instead.
4. Two filters presented as facets - one by tag, one by publish date period - that a visitor can click to narrow results, with accurate counts.
5. A "quick reads only" filter driven by the reading time value from Task 6.

7b. Site-wide search

Build a **separate** search page that searches the whole site and returns **mixed content types in a single result list** - blog posts, other pages, and uploaded documents - each with a title, an excerpt and a working link, regardless of type.

1. At least one PDF or Word document must appear in these results.
2. Provide a filter to narrow results by type: all, pages, blog posts, or documents.

7c. Boosting

Influence result ranking in **these distinct ways**:

1. Rank posts published within the last 90 days above older posts, and posts tagged **optimizely** above untagged ones - using the values computed in Task 6.

2. Configure the relative importance of title, content, summary and document content for the site-wide search of 7b, and include a screenshot showing the effect.

Additionally, promote a chosen blog post to the top of results for the search term **optimizely**, configured by an administrator rather than hard-coded.

TASK 8 - Author URLs

Author profiles are placed in a folder (For all sites). Blog pages will show who author is and have a link to navigate to author profile page:

```
/blog/author/jane-doe
```

The jane-doe matches with the **slug** in Jane Doe **AuthorProfileBlock**

1. The URL must resolve to a rendered author profile page showing that author's details.
 2. The page must list that author's blog posts.
 3. An unrecognised author must produce a clean **404**. A server error is a failure.
 4. The author page must show the correct site header and footer according to the rules in Task 4.
 5. *Bonus*: support paging at **/blog/author/jane-doe/page-2**.
-